How Secure Is Your Base? A Recent Multi-Scanner Census of Linux Operating-System Images on Docker Hub

Anonymous

[1]Anonymous Institution

Abstract. *Every container starts from an operating-system (OS) base image, yet these are usually measured with a single scanner. We present a recent multi-scanner census of Docker Hub's Linux OS base images:* ***5,606*** *unique* `amd64` *images across* ***20*** *repositories, scanned with* ***14*** *open-source tools. We find that vulnerability posture varies by an order of magnitude across distributions and is driven by image age and a distribution's patch cadence rather than by image size or popularity; and that about one in five historically-published images can no longer be pulled by a modern Docker (legacy manifest schema). As a secondary result, the four package-vulnerability engines barely agree, so the posture a single tool reports is largely a property of that tool. We publicly release this multi-scanner dataset, to our knowledge the first focused on OS base images, together with the full pipeline.*

1. Introduction

Containers are now a default unit of software delivery: Docker reports more than 20 million developers[1] and, in its 2025 survey, 92% of IT professionals using containers[2]. Underneath them sits Docker Hub, the largest public registry, with on the order of 8.3 million image repositories and hundreds of billions of cumulative pulls[3]. Every one of those images ultimately begins with a single line, `FROM` an operating-system (OS) base image, and a handful of bases are the common root of nearly all of them: `alpine`, `ubuntu`, `debian` and `busybox` each record more than a billion pulls, and Debian, Alpine and Ubuntu dominate the base-image choice of the ecosystem[4]. Because a flaw in a base image is inherited, unchanged, by every image built on it [Shu et al. 2017], the security posture of these few OS bases is a property the whole ecosystem inherits.

That inherited posture is poor and consequential. Base images are the root of the software supply chain, so their known vulnerabilities are a supply-chain risk in their own right, alongside the malicious-injection attacks that dominate headlines (projected to cost USD 60 billion in 2025[5] and predicted by Gartner to hit 45% of organisations by 2025[6]). And the inherited risk is large: popular Docker Hub images carry hundreds of known vulnerabilities on average, many of them years old[7], against a backdrop of record CVE disclosure (over 48,000 CVEs in 2025)[8]. Worse, some of the most-pulled OS bases are

[1]Docker, 2024 report
[2]Docker, 2025 survey
[3]Docker Index
[4]Anchore, 2024
[5]Cybersecurity Ventures
[6]Gartner, 2022
[7]NetRise, 2024
[8]CVE Details

no longer maintained (every official `centos` tag reached end-of-life in June 2024) yet remain in heavy use.

Despite this central role, the security posture of OS base images has usually been characterised with a *single* scanner and on samples that age quickly: the large-scale Docker Hub measurements have typically used one detector [Shu et al. 2017, Zerouali et al. 2019, Liu et al. 2020, Shi et al. 2025], and the studies that *do* compare scanners have done so on focused samples [Mills et al. 2023, Boles et al. 2024]. No recent work measures the Linux OS base images specifically, with a heterogeneous multi-scanner battery, as a current census. This paper fills that gap.

Contributions. We present (i) a recent multi-scanner census of Docker Hub's Linux OS base images (Section 3); (ii) a per-distribution posture, a staleness-vulnerability relationship, and the finding that vulnerability load is driven by age rather than image size or popularity (Sections 4.1, 4.2, 4.5); (iii) an image-longevity finding (Section 4.4); (iv) as a secondary result, a four-engine divergence measurement on the OS-package layer (Section 4.3); and (v) a public multi-scanner dataset, to our knowledge the first focused on OS base images: the per-image findings of all 14 scanners over the census, released with the full pipeline so the corpus can be re-analysed under other metrics or scanners without repeating the costly scan.

2. Related Work

Table 1 positions our work against prior measurements along six axes.

Large-scale measurements. A rich line of work has charted the security of Docker Hub. [Shu et al. 2017] introduced the first large-scale study, scanning 356,218 images and showing that vulnerabilities propagate from a base image to everything built on it. [Liu et al. 2020] extended the scale to 2.2 million images; [Zerouali et al. 2019] introduced the notion of *technical lag* on 7,380 Debian-based images, later generalised to a multi-dimensional analysis over 140,000 images [Zerouali et al. 2021]. [Wist et al. 2021] analysed 2,500 images across image classes, [Ibrahim et al. 2020] examined how images for the same system differ across base distributions, and the recent [Shi et al. 2025] ranked influential images by pull count at the scale of the full registry. Most relevant to us, [Haque et al. 2022] characterised the exploitability and impact of vulnerabilities in 261 official base images and reported a result we revisit in Section 4.5: highly-exploitable vulnerabilities are *more* frequent in minimal images such as Alpine, while larger distributions carry the higher-impact ones. These studies established the prevalence and propagation of vulnerabilities in the ecosystem; each draws on a single vulnerability detector, and our multi-scanner design complements them by also quantifying how much the measured posture depends on the chosen scanner.

Inter-scanner divergence. A second line studies how much a scan result depends on the tool used. [Imtiaz et al. 2021] compared nine software-composition-analysis tools and found counts spanning an order of magnitude, and [Javed et al. 2021a, Javed et al. 2021b] report comparable variation among container scanners. Side-by-side comparisons on focused samples include [Kaur et al. 2021] (four scanners, 44 images), [Mills et al. 2023] (six, 380), and [Churakova et al. 2026] (seven scanner and VEX tools, 48 images; counts from 191 to 18,680, best-pair Trivy and Grype agreeing about 69%); closest to us, [Boles et al. 2024] compared Trivy and Grype on 927 Docker Official Linux images

Table 1. Prior OS/base-image and container-scanner measurements.

Study	Images	Tools	Dims	Source	OS	X-sc.
[Shu et al. 2017]	356,218	1	V	Hub crawl	✗	✗
[Zerouali et al. 2019]	7,380	1‡	V	Hub, Debian	✓	✗
[Liu et al. 2020]	2.2 M*	1	V	Hub crawl	✗	✗
[Ibrahim et al. 2020]	936	1	V	Hub, per-sys	✗	✗
[Kaur et al. 2021]	44	4	V	curated	✗	✓
[Zerouali et al. 2021]	140,498	1‡	V	Hub, Debian	✓	✗
[Haque et al. 2022]	261	1	V	Hub, official	✓	✗
[Dahlmanns et al. 2023]	337,171	1	S	Hub crawl	✗	✗
[Malhotra et al. 2023]	n/r	4	V	Hub off./ver.	✗	✓
[Mills et al. 2023]	380	6	V	Hub categories	✗	✓
[Boles et al. 2024]	927	2	V	Hub official	✓	✓
[Kawaguchi et al. 2024]	3,514	2	B	Hub popular	✗	✓
[Bufalino et al. 2025]	20†	7	V	Hub Deb/Alp	✓	✓
[Shi et al. 2025]	33,952*	1	V,S,C,M	Hub pull-rank	✗	✗
[Churakova et al. 2026]	48	7	V	Hub curated	✗	✓
This work	**5,606**	**14**	**V,B,S,C,M**	**Hub category**	✓	✓

Dims: V vuln, B SBOM, S secrets, C config, M malware. X-sc.: cross-scanner. ∗ vulnerability scan on a subset; † top-20 Debian/Alpine image families; ‡ Debian Security Tracker (metadata), not a scanner.

and found them agreeing on the count for only 15%. On the OS-package layer, [Bufalino et al. 2025] report that scanners are largely incompatible owing to package-identifier mismatches, and [Zhou et al. 2026, Rosso et al. 2026] document high false-positive rates and silent tool failures in SBOM-based scanning. Our work brings this multi-scanner lens to the OS-base layer at census scale and relates the divergence to image age and distribution.

Gap. Building on this body of work, we are not aware of a recent study that measures the Linux OS base images of Docker Hub specifically with a heterogeneous multi-scanner battery as a current census, nor one that quantifies the prevalence and popularity of *end-of-life* OS images that remain in heavy use. This paper addresses both.

3. Methodology

Threat model and scope. We measure the security posture an OS base image presents to the two parties who choose and run it: the developer who writes `FROM` and the operator who pulls a digest. The risk we characterise is *inherited* exposure, the latent, known-vulnerable surface that an unchanged base propagates to every image built on it, as it would appear to a posture scan at selection or pull time. We do not model an active attacker injecting malicious code (the domain of typo-squatting and malicious-package work); the legacy-schema and end-of-life findings (Section 4.4) bear on supply-chain integrity and the reproducibility of historical builds rather than on remote exploitation. "Insecure" here means *carries known vulnerabilities a defender would have to triage*, not *remotely exploitable*.

Corpus. We take as our corpus the operating-system base images Docker Hub lists under its *Operating systems* category: **20** repositories with an `amd64` image (`alpine`, `ubuntu`, `debian`, `centos`, `busybox`, `amazonlinux`, `fedora`, `oraclelinux`, `rockylinux`, `almalinux`, `photon`, `archlinux`, `tumbleweed`, `leap`, `kali-rolling`, `mageia`, `alt`, `cirros`, `clearlinux`, `sl`). Deduplicating tags by `amd64` content digest (e.g. `24.04`, `noble` and `latest` are one image) leaves **5,606** unique images, the unit of measurement.

Scanner battery. An OS base image is a static root filesystem with no application source code and no running service, so we select the static-analysis axes that apply to such an artifact and run *14* open-source scanners.[9] *SBOM*: Syft and cdxgen. *Package vulnerabilities (SCA)*: Trivy, Grype, OSV-Scanner and Clair. *Image hardening*: Dockle and Checkov. *Secrets*: TruffleHog, Gitleaks, detect-secrets and Whispers. *Malware/IOC*: ClamAV and YARAHunter. We exclude the SAST, dynamic (DAST) and network axes, which need application source or a running service a base image lacks. Each image is exported once to a tar archive that all 14 scanners read, so result differences reflect the scanner rather than the pull, and raw outputs are kept verbatim.

4. Results

4.1. RQ1: Security posture by distribution

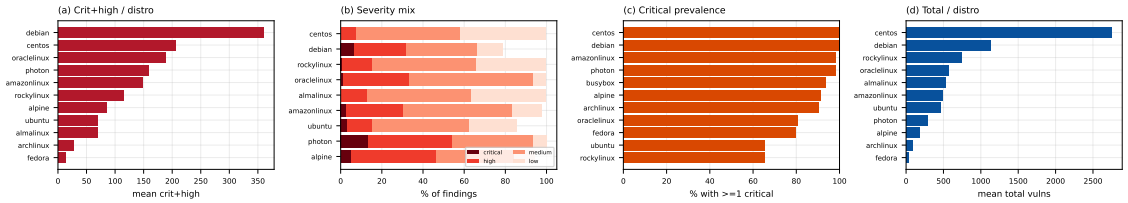

Figure 1. (a) mean critical+high per distribution; (b) severity mix; (c) critical prevalence; (d) mean total vulnerabilities.

Critical- and high-severity findings vary by an order of magnitude across distributions (Figure 1, Table 2): Debian and the RHEL family (CentOS, OracleLinux, AmazonLinux, RockyLinux) carry the most, while minimal and rolling-release bases (BusyBox, Arch, openSUSE Leap) carry the fewest. Within-distribution variance is large (Debian's SD exceeds its mean), so these are tendencies, not tight estimates. Crucially, package count does not track the ordering (Table 2): Fedora and AlmaLinux are package-heavy yet mid-to-low in severe findings, while one-package BusyBox is near the floor (Section 4.5).

4.2. RQ2: Staleness vs. vulnerability

Vulnerability count tends to rise with image age (Figure 2(a)). Images rebuilt within the last six months carry a mean of ≈255 total vulnerability findings, climbing through ≈273 (one to two years) and ≈457 (two to four years) to ≈692 for images not rebuilt in over four years. The per-image rank correlation is positive but modest (Spearman $\rho = 0.16$, $p < 0.001$), reflecting wide within-bucket variance even as the bucketed means rise monotonically (the technical lag of [Zerouali et al. 2019], here across 20 distributions).

Table 2. Per-distribution posture (repositories with $n \geq 10$ images), sorted by mean critical+high per image. *age*: median days since last push.

Distro	n	cr+hi	total	pkgs	%cr	age
debian	389	360	1130	85	100	164
centos	10	206	2754	161	100	2078
oraclelinux	26	189	568	169	81	1019
photon	363	158	290	38	98	1025
amazonlinux	312	148	485	113	98	1113
rockylinux	95	115	741	134	65	1101
alpine	181	84	181	14	91	1381
ubuntu	270	70	459	94	66	1014
almalinux	234	69	522	123	51	344
archlinux	441	27	93	134	90	619
fedora	20	13	31	155	80	1184
busybox	46	8	13	1	93	1870
leap	10	3	8	130	10	1439

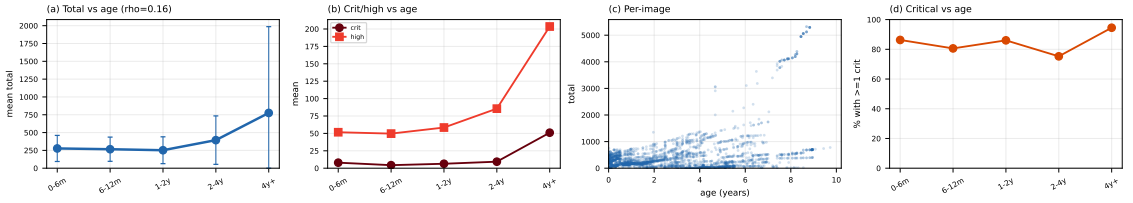

Figure 2. (a) mean total by age (±1 SD); (b) critical and high by age; (c) per-image age vs. total; (d) critical prevalence by age.

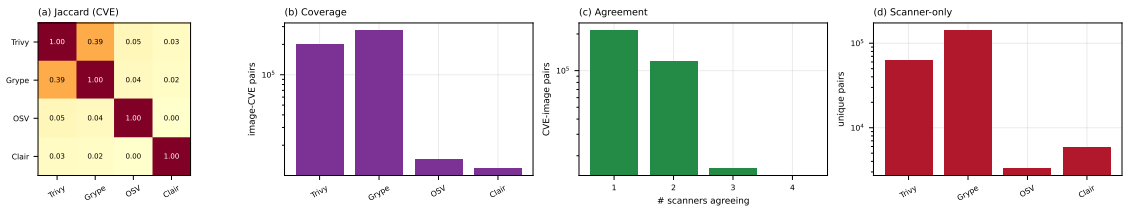

Figure 3. (a) pairwise CVE Jaccard; (b) CVEs per engine; (c) CVEs by number of agreeing engines; (d) engine-only CVEs.

4.3. RQ3: Inter-scanner divergence on OS packages

The four package-vulnerability engines barely agree (Figure 3). We compare at the (image, CVE) level because the engines name packages differently (Clair reports source packages, Trivy and Grype binary packages), so a package-level key would understate agreement. Even so, the best-agreeing pair is Trivy and Grype, at a Jaccard of just **0.35**; every other pair is below 0.05, and OSV-Scanner and Clair are near-disjoint from the rest (OSV∩Clair $= 0$). The engines differ sharply in scope: Grype and Trivy report by far the most image-CVE pairs (196k and 148k), OSV-Scanner targets language-ecosystem advisories (9k) and Clair maps only specific distributions to its feeds (8k, and zero on Rocky and AlmaLinux, which it does not map to the RHEL feeds). The reported security posture of an OS image is thus, to a large degree, a property of the chosen scanner.

[9]Each scanner runs from a version-pinned container image; the exact versions are recorded in the released configuration.

4.4. RQ4: End-of-life and un-pullable images

Two longevity hazards surface. First, about **one in five** of the digest-pinned OS images fail to pull on a current Docker Engine with `manifest schema unsupported`: they are old images in the deprecated Docker image-manifest schema 1, whose support modern Docker removed. The digests still exist on Docker Hub, but the images are no longer installable, and the rate varies sharply by distribution (Figure 4): over half of the `centos` and `busybox` digests and about a third of `ubuntu` are affected, against under 3% for `debian`, `archlinux` and `rockylinux`. Second, end-of-life distributions remain in heavy use: every `centos` tag has been unmaintained since June 2024, yet the repository still records over a billion pulls. To our knowledge neither the prevalence of legacy-schema OS images nor the popularity of EOL bases has been measured before.

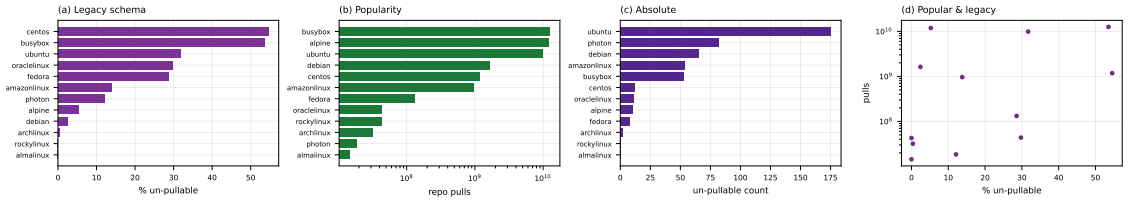

Figure 4. (a) un-pullable rate by distribution; (b) repository pulls; (c) un-pullable count; (d) rate vs. pulls.

4.5. RQ5: What drives the vulnerability load?

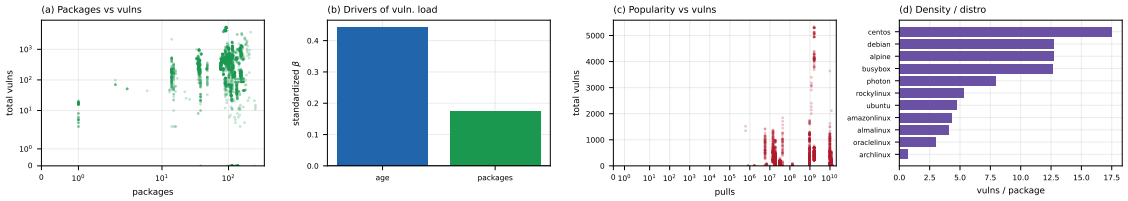

Figure 5. (a) packages vs. total; (b) standardized regression coefficients (age vs. packages); (c) popularity (pulls) vs. total; (d) vulnerabilities per package by distribution.

The intuition that a smaller image is a safer one does not hold cleanly (Figure 5(a)). The relationship between package count and vulnerability count is not monotonic: Busy-Box, with a single package, carries only a handful of findings, but Mageia, with around 250 packages, carries about a dozen, while Debian carries hundreds of critical and high findings with fewer packages than Mageia. This refines [Haque et al. 2022]: on our multi-scanner data the vulnerability count is governed more by a distribution's advisory coverage and patch cadence than by image size. A multiple regression makes the ranking explicit (Figure 5(b)): with both predictors standardized, image age dominates ($\beta \approx 0.44$) over package count ($\beta \approx 0.17$), and the partial correlations agree (0.44 vs. 0.19); the vulnerability load tracks patch cadence, not image size. Popularity offers no protection either: a repository's pull count is essentially uncorrelated with its vulnerability count (Spearman $\rho \approx 0$, Figure 5(c)), so the most-pulled bases are no safer than the rest.

A four-predictor model confirms the ordering. Regressing total vulnerabilities on standardized image age, package count, image size and pull count gives $\beta_{\text{age}} = +0.46$, $\beta_{\text{packages}} = +0.27$, $\beta_{\text{size}} = -0.22$ and $\beta_{\text{pulls}} = -0.05$ (n=2,421): age dominates, and the

negative size coefficient is a suppressor effect from the near-collinearity of size and package count (Spearman 0.89; Table 3). The full rank-correlation matrix (Table 3) makes the picture explicit: age is the only predictor positively associated with the vulnerability load, while size and popularity are flat or slightly negative. This is the multivariate counterpart to RQ1's observation that a distribution's patch cadence, not its girth, sets its posture.

Table 3. Spearman rank correlations among per-image metrics (n=2,421).

	vulns	pkgs	age	size	pulls
vulns	1.00	0.06	0.18	−0.03	−0.00
pkgs		1.00	−0.15	0.89	−0.35
age			1.00	−0.09	0.12
size				1.00	−0.32
pulls					1.00

A single-level regression is, however, confounded by distribution: older bases (the RHEL family) are also larger and staler. A linear mixed model with a random intercept per distribution disentangles the two. Distribution identity alone accounts for about half the variance (intraclass correlation 0.53), and once it is absorbed, package count becomes a *strong* within-distribution driver ($\beta = 0.51$), comparable to age ($\beta = 0.42$; both $p < 10^{-29}$, $n = 2{,}397$, 13 distributions). The naive "age beats size" reading therefore overstates: most of it is between-distribution variation, while *within* a distribution more packages do mean more vulnerabilities. The actionable lever is the distribution; within it, both currency and minimalism help.

4.6. Beyond vulnerabilities: secrets, misconfiguration, malware

The battery also covers four non-vulnerability axes. *Misconfiguration:* Dockle and Checkov flag at least one CIS-style issue in essentially every image (missing `HEALTHCHECK`, content-trust disabled, running as `root`); these are structural properties, so the near-universal rate is robust. *Secrets:* TruffleHog and Gitleaks raise at least one detection in **72%** of images, but these are raw hits. We manually reviewed a seeded random sample of **1,100** of the 26,892 detections (stratified by scanner, 95% CI $\pm 3\%$), reading each one rather than filtering by rule: **none** was a real credential (true-positive rate $\leq 0.35\%$, 95% Wilson CI). Every hit was a test key (e.g. GnuTLS fixtures in `libgnutls.so`), a package checksum matched as a token, a documentation example, a public key, or a C identifier (`TPM2B_ENCRYPTED_SECRET`). The validated secret rate for OS base images is thus effectively zero (even below [Dahlmanns et al. 2023]'s 8.5% and Dr. Docker's $\sim$0.7% valid [Shi et al. 2025]), because real secrets live in application layers, not the base. The labelled sample is released for re-use. *Malware/IOC:* ClamAV reports zero detections; YARA-Hunter matches 76% of images, but we manually reviewed a seeded random sample of 1,100 of its 13,348 matches and found **zero** real malware (true-positive rate $\leq 0.35\%$, 95% Wilson CI). Every match was a YARA rule (several written for memory dumps) firing on a benign OS artifact: an Emotet byte-pattern in the BusyBox binary, a RAT rule matching the word "cerberus" in the IEEE OUI list, rootkit rules matching kernel syscall names in canonical headers. This agrees with ClamAV's zero and Dr. Docker's 24-in-33,952. Across all three non-vulnerability axes, then, the raw single-tool rate is high (72–100%) but the validated rate collapses to near zero for secrets and malware: as in RQ3, the headline number is

largely a property of the detector, and reading the findings is what separates signal from noise. The labelled malware sample is released alongside the secrets one.

4.7. Reproducing prior analyses

We reproduce concrete analyses from prior work on *our* OS-base corpus (Table 4); their own corpora range from the whole registry to a single distribution (the *OS?* column marks the OS-base-focused ones), yet the direction holds in every case, and the multi-scanner setting refines the minimal-image and scanner-agreement findings.

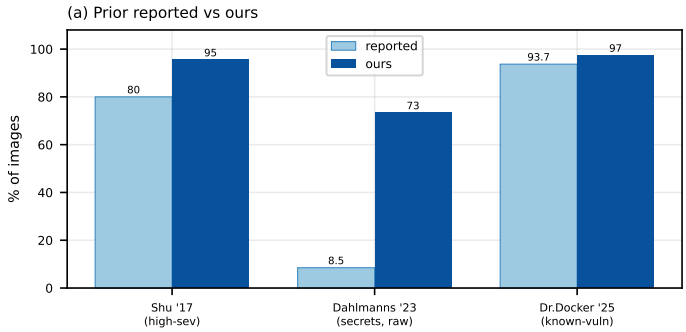

Figure 6. Prior reported vs. ours for the %-of-images metrics we reproduce (Shu high-severity, Dahlmanns secrets raw, Dr. Docker known-vulnerability).

Three patterns emerge (Figure 6). *Prevalence replicates, and upward*: the share of images carrying a high-severity vulnerability rises from [Shu et al. 2017]'s >80% (2017) to our 97%, and [Shi et al. 2025]'s 93.7% known-vulnerability rate to 98.7%, consistent with the secular growth in disclosed CVEs and the aging of un-rebuilt bases (RQ2). *Staleness and the minimal-image question reproduce in direction but sharpen in mechanism*: as in [Zerouali et al. 2019] vulnerabilities rise with age, and as in [Haque et al. 2022] a minimal image is not reliably safer, but our four-predictor model (Section 4.5) attributes the load to patch cadence rather than to size or popularity. *Divergence is larger on the OS layer than two-tool studies suggest*: where [Boles et al. 2024] found Trivy and Grype agreeing on the count for only 15% of images, our CVE-level Jaccard of 0.35 and the near-disjoint OSV-Scanner and Clair sets (RQ3) show the disagreement deepens once four engines, rather than two, are compared. The reproductions thus serve a dual role: they validate the pipeline against established results, and they quantify how much a single-scanner, single-distribution lens understates the ecosystem-wide picture.

5. Discussion and conclusion

Implications. Three actionable readings follow from the data. (i) The distribution is the dominant lever (Section 4.5): pin to a maintained, fast-patching base and rebuild on a cadence rather than chase a "minimal" tag, since image size alone does not predict exposure. (ii) Because a single scanner's count is largely a property of that scanner (Section 4.3), treat a one-tool number as a lower bound and reconcile at least two engines with complementary feeds, a distro-mapped one (Clair) and an advisory-based one (Trivy or Grype); our released multi-scanner dataset is a ready reconciliation baseline. (iii) The high raw but near-zero validated rates for secrets and malware (Section 4.6) argue for validation-in-the-loop before acting on any non-vulnerability finding.

Table 4. Prior analyses reproduced on our OS-base corpus.

Study	Metric reproduced	OS?	Reported	Ours
[Shu et al. 2017]	images with a high-severity vuln	✗	>80%	97%*
[Zerouali et al. 2019]	vulnerabilities vs. image age	✓	rises with age	≈255→692 (ρ=0.16)
[Ibrahim et al. 2020]	spread across base distributions	✗	varies	10× range
[Wist et al. 2021]	where severe vulns concentrate	✗	language ecosys.	OS-package layer
[Wist et al. 2021]	official imgs w/ crit or high	✗	45.9%	96%
[Haque et al. 2022]	is a minimal image safer?	✓	no	no (non-monotonic)
[Haque et al. 2022]	base images with ≥1 vuln	✓	72.8%	97%
[Dahlmanns et al. 2023]	images carrying secrets	✗	8.5% valid	72% raw, 0% valid
[Mills et al. 2023]	vulnerabilities vs. time; OS effect	✗	rise; Debian ≫	rise; Debian top
[Boles et al. 2024]	scanner divergence on OS images	✓	agree 15%	Jaccard 0.35
[Churakova et al. 2026]	Trivy/Grype CVE Jaccard	✗	0.76	0.35
[Bufalino et al. 2025]	CVE-count spread across tools	✓	order of mag.	12k to 275k
[Shi et al. 2025]	known-vulnerability prevalence	✗	93.7%	98.7%

OS?: prior study's corpus is OS base images. ∗ critical or high (Shu reports high-severity); ρ: Spearman.

Limitations. Small repositories carry wide within-distribution variance, so per-distribution means are tendencies, not tight estimates. We report raw scanner output without adjudicating false positives or negatives; the divergence of RQ3 implies no engine is ground truth, and severity rests on CVSS scores that can overstate risk without VEX or reachability data [Zhou et al. 2026]. We scan `linux/amd64` only, characterise each image as of its scan time, and cover only the *Operating systems* category; the released dataset lets others re-run the analysis under other choices. **Conclusion.** We measured the security posture of Docker Hub's Linux OS base images with 14 scanners rather than one. Posture varies by an order of magnitude across distributions and is driven by image age rather than image size or popularity; and about a fifth of historically-published images can no longer be installed by a modern Docker while end-of-life bases stay heavily pulled. Secondarily, the four package-vulnerability engines agree on almost nothing, so a single tool's count is largely a property of that tool. We publicly release the consolidated multi-scanner dataset and pipeline[10] so the measurement can be reproduced, audited, and extended.

References

Boles, B. et al. (2024). Deciphering discrepancies: A comparative analysis of Docker image security. In *SCAM*. IEEE.

Bufalino, J. et al. (2025). SBOMproof: Beyond alleged SBOM compliance for supply chain security of container images.

Churakova, Y. et al. (2026). Vexed by VEX tools: Consistency evaluation of container vulnerability scanners. *arXiv preprint arXiv:2503.14388*. version 3.

Dahlmanns, M. et al. (2023). Secrets revealed in container images: An internet-wide study on occurrence and impact. In *ASIA CCS '23*, pages 797–811. ACM.

Haque, M. U. et al. (2022). Well begun is half done: An empirical study of exploitability and impact of base-image vulnerabilities. In *SANER*, pages 1100–1111. IEEE.

[10]Released on acceptance.

Ibrahim, M. H. et al. (2020). Too many images on DockerHub! how different are images for the same system? *Empirical Softw. Eng.*, 25(5):4250–4281.

Imtiaz, N. et al. (2021). A comparative study of vulnerability reporting by software composition analysis tools. In *ESEM*.

Javed, O. et al. (2021a). An evaluation of container security vulnerability detection tools. In *ICCBDC*, pages 95–101. ACM.

Javed, O. et al. (2021b). Understanding the quality of container security vulnerability detection tools. *arXiv preprint arXiv:2101.03844*.

Kaur, B. et al. (2021). An analysis of security vulnerabilities in container images for scientific data analysis. *GigaScience*, 10(6):giab025.

Kawaguchi, N. et al. (2024). Understanding the effectiveness of SBOM generation tools for manually installed packages in Docker containers. *JISIS*, 14(3):191–212.

Liu, P. et al. (2020). Understanding the security risks of Docker Hub. In *Computer Security – ESORICS 2020*, volume 12308 of *Lecture Notes in Computer Science*, pages 257–276. Springer.

Malhotra, R. et al. (2023). Vulnerability analysis of Docker Hub official and verified images. In *SOSE*, pages 150–155. IEEE.

Mills, A. et al. (2023). Longitudinal risk-based security assessment of docker software container images. *Comput. Secur.*, 135:103478.

Rosso, M. et al. (2026). A practical solution to systematically monitor inconsistencies in SBOM-based vulnerability scanners. In *SAC '26*. ACM.

Shi, H. et al. (2025). Dr. Docker: A large-scale security measurement of Docker image ecosystem. In *WWW '25*. ACM.

Shu, R. et al. (2017). A study of security vulnerabilities on Docker Hub. In *CODASPY '17*, pages 269–280. ACM.

Wist, K. et al. (2021). Vulnerability analysis of 2500 Docker Hub images. In *Advances in Security, Networks, and Internet of Things*, Transactions on Computational Science and Computational Intelligence, pages 307–327. Springer.

Zerouali, A. et al. (2019). On the relation between outdated Docker containers, severity vulnerabilities, and bugs. In *SANER 2019*, pages 491–501. IEEE.

Zerouali, A. et al. (2021). A multi-dimensional analysis of technical lag in Debian-based Docker images. *Empirical Softw. Eng.*, 26(2):19.

Zhou, L. et al. (2026). A reality check on SBOM-based vulnerability management: An empirical study and a path forward. In *CODASPY '26*. ACM.